

We will solve the problem in an expected way: by finding some shortest paths in some suitably compressed state space graphs. But in order to get there we first need to analyze the problem to actually determine what those graphs look like and how to construct them efficiently and without writing too much code. A nice property of the problem is that the more work you do before you start coding, the simpler the code can be.

Equivalence classes

If we say that two numbers are equivalent if one can be rearranged to the other, we can now divide all positive integers into equivalence classes. An important observation about them is that the number of these equivalence classes grows *only polynomially* with the number of non-zero digits: there are $\binom{d+8}{8}$ numbers with d digits, all of them non-zero and sorted in non-decreasing order. E.g., for $d = 12$ there are 125 970 and for $d = 18$ there are still only 1 562 275 such equivalence classes.

These will not exactly be the states of our search, but for now it's close enough to have this general idea.

Width equals one is enough

Suppose you have any solution such that its width is $w > 1$.

What happens when we replace each “ $+x$ ” resize with x consecutive $+1$ s, and each “ $-x$ ” resize with x consecutive -1 s?

Each original resize turns into at most w new resizes, so the length increases at most w times. At the same time, the width decreases exactly w times: from w to 1. Hence, the new area is at most equal to the old area.

This implies that **there is always an optimal solution with width either 0 or 1**.

Having width = area = 0 is only possible iff a and b have the same multiset of non-zero digits. This is easily handled as a special case. Hence, in the general case that's left we always have an optimal solution with width exactly 1.

Optional observation: answer is always small

For one possible loose upper bound we can see that with width=9 we can use a resize to change the last digit of a number to any other value, and as we can rearrange between resizes, a sequence of at most d such edits can turn any d -digit number into any other d -digit number, so the answer is always at most $9d$.

If we are more careful about how we change the digits (e.g., going from 2 to 6 by doing four increments but going from 2 to 8 by changing the 2 to 0 and later producing a 9 and decrementing it to 8), we can further decrease the upper bound to roughly $4d$.

Towards actual states and transitions

Given an equivalence class as mentioned above we can easily enumerate all transitions — all effects of incrementing or decrementing a number from that class.

E.g., suppose our equivalence class is “numbers with non-zero digits 11468”. Some of the numbers from this class include 11468, 614018, and 1004618. What do all of these have in common? In all of them the “ $+1$ ” operation changes the units digit to 9, and thus the new equivalence class is 11469. This would be the same for all other numbers where 8 is the units digit. And the same observation can be made about the “ -1 ” operation: the only change in all of these numbers is that the 8 turns into a 7.

For each equivalence class most of the transitions look like the ones shown above. There are only two exceptions: decrementing a number that ends in one or more 0s, and its inverse — incrementing a number that ends in one or more 9s.

Decrementing a number that ends in some 0s decrements its last *non-zero* digit (i.e., does the same thing it would if the zeros weren't present), but additionally it turns all those trailing 0s into 9s.

Conclusion: whenever we decrement the current number, we can just consider the cases when the last digit is non-zero, but additionally in each of them we can then add arbitrarily many 9s to the state.

Conversely, incrementing a number that ends in a sequence of 9s turns them into 0s and then increments the last non-nine digit. Hence, whenever we increment the current number, we can just consider the cases in which we increment a non-9 digit, but additionally in each of them we can remove arbitrarily many 9s from the state.

Actual states

We store the exact count of digits from 1 to 8 we have. In each state we can have arbitrarily many 0s. For 9s we will store two counts: the smallest and the largest number of 9s we can have. (All intermediate counts are clearly also possible. The upper bound can be infinite — in fact, it becomes and stays infinite as soon as we use a decrement at least once.)

Lower bound

Let's look at digits with values 1-8 in a number. Each resize (i.e., an increment or a decrement) changes the value of at most one of them by one.

We can now consider a simplified problem: We are maintaining a multiset of digits. We can add and remove 0s and 9s completely for free, and changing the value of any one digit by 1 costs 1. In this setting we want to transform one multiset to another.

Clearly, the solution to this new problem is always a lower bound on the solution to our problem.

On the other hand, this simplified problem is reasonably easy to solve efficiently. The key observation is that there is always an optimal solution such that the digits in the multiset never change their relative order. This is because if you start with two digits $a < b$ in your multiset and eventually you change a to a' and b to b' such that $a' > b'$, you could have instead changed a to b' and b to a' and it would preserve their relative order and require at most as many steps.

A-star search

As a solution that solves everything except the largest subproblem we implemented A-star with the above heuristic. (Instead of just running BFS we run modified Dijkstra where the states in the queue are sorted by the sum of their actual distance from the start and a lower bound on their distance to the finish.)

Essentially, this solution solves most of the problem greedily and then searches through all ways how to deal with the pesky 0s and 9s to pick the best one.

Greedy solution

Before you try to solve the problem fully greedily, it is highly advisable to implement at least one slower solution so that you can stress-test your implementation to make sure that you didn't miss any case.